LEAN: Loss-Engineered Modulation of the Weight-Sensitivity Distribution

Supplementary Material: Code, Hardware and Reproduction Manual

Anonymous Submission

Anonymous code repository (artifact):
https://anonymous.4open.science/r/LEAN
Hosted via Anonymous GitHub: https://anonymous.4open.science

This document goes with the anonymized code for **LEAN**. It explains what the code does, how the repository is organized, which script produced each table and figure in the paper, and how to run the three parts yourself: the training objective and the attacks on vision models, the language-model experiments, and the hardware loader with its RTL simulation, ASIC synthesis and FPGA implementation. We tried to keep every experiment one command away; where a run takes hours, the script writes a log that can be followed while it runs.

Contents

1 Overview **1**

2 Repository layout **1**
2.1 Environment 2

3 Core scripts **2**

4 Paper table and figure map **3**

5 One model end to end **4**

6 Language models **4**

7 Hardware **4**

8 Energy model **5**

9 Notes **5**

1 Overview

LEAN is a loss term. The damage a bit flip does to a stored weight is the product of the weight's gradient and the step the flip takes in the storage format, and **LEAN** regularizes both: a range term

bounds the flip step of every layer, and a tail term, the mean sensitivity of the top 1% of weights differentiated through the gradient, lowers the gradient where it is largest. One coefficient λ sets how far the sensitive tail moves. The term is added to the loss of an ordinary ten-epoch fine-tune or of the last epochs of training; the architecture, the quantizer and the inference hardware are unchanged.

Everything lives in one flat `code/` directory so that the scripts can import each other without packaging: the objective for CNNs and for language models, the sensitivity measurement, the real-flip check, the four attacks (progressive bit search, T-BFA, AttentionBreaker, SilentStriker), the selective code with the real BCH codec and the key-aware attacker, the sensitivity-ordered 8/6/4/3-bit storage format and the NVFP4 mode, the five published training-time defenses and the two quantization baselines reproduced under the paper's protocol, and the shell scripts we actually ran for every table. `hw/` holds the weight loader and all hardware flows, `energy/` the Timeloop and Accelergy inputs.

2 Repository layout

```
code/     Python and shell scripts (run everything from inside code/)
hw/       loader RTL, testbenches, Design Compiler, Vivado FPGA and Zynq SoC flows
energy/   Timeloop + Accelergy architecture, constraints, workloads, run_all.sh
README.md, requirements.txt, LICENSE (MIT)
```

2.1 Environment

```
conda create -n lean python=3.10 && conda activate lean
pip install -r requirements.txt
```

CUDA GPUs are assumed: 24 GB is enough for all vision experiments and the 0.5B to 1.5B language models; the 7B and 8B models run in bfloat16 on one 48 GB GPU per attack. CIFAR-10/100 are downloaded by torchvision. ImageNet experiments use the 50k validation set split into 40k fine-tune and 10k held-out images (the paper states every ImageNet accuracy on that held-out set), expected at `./imagenet-val`. Language models are loaded from the Hugging Face hub. Clean checkpoints live under `artifacts/models/<dataset>/<arch>/model_float32.pth`; CIFAR baselines are trained with `gen_models.py`, ImageNet baselines are the torchvision weights.

3 Core scripts

`lean_sensreg.py` the **LEAN** objective for CNNs. Sub-commands: `train` (fine-tune with `--reg tail|clip|both|none`, `--lam`, `--tail-frac`, `--tail-every`, `--clip-q`, `--clip-w`), `hist` (HIGH-/MID/LOW shares against the baseline's thresholds), `plot`, `ptq`, `flip` (real sign-bit flips, targeted or random), `knob` (the λ sweep), `figure`.

`lean_sensreg_llm.py`, `lean_llm_train_fd3.py`, `lean_llm_train_big.py`, `lean_llm_train_pcstep.py` the same objective on the linear layers of language models (WikiText-2, one epoch); the finite-difference form of the tail term for 1.5B and larger models; the per-channel-step variant.

`lean_pcstep.py`, `lean_train_pcstep_cnn.py`, `lean_hist_pc.py`, `lean_truedamage_pc.py` per-channel INT8 step for the tail term, the per-channel sensitivity measurement and real-flip damage.

`lean_truedamage.py`, `lean_train_gradnorm.py` the real-flip check of Table 2 and the plain gradient-norm penalty.

`pbs_four.py` progressive bit search on the INT8 model under no code, random code and the selective code (`--protect none|uniform|selcode`, `--covs`, `--flips`); `lean_pbs_codec.py` runs the real BCH codec in the loop and the key-aware attacker; `lean_pbs_codec_pc.py`, `_dw.py`, `_clamp.py` are the per-channel, depthwise-coded and activation-clamp variants.

`lean_tbfa.py`, `lean_tbfa_all.py` the targeted attack on held-out data.

`lean_train_hessian.py`, `lean_train_pc_wcat.py`, `lean_train_far.py`, `lean_train_lsq.py` Hessian-aware training, Piece-wise Clustering, WCAT, Forget-and-Rewire, LSQ and KURE, same data, epochs and initialization as **LEAN**.

`lean_train_inset.py`, `_inset2.py`, `_inset3.py` **LEAN** switched on in the last ten epochs of a 200-epoch run (Appendix C).

`lean_train_qat_group5.py`, `lean_train_qat_mp9.py`, `lean_eval_mp.py` the sensitivity-ordered 8/6/4/3-bit format with per-block (hardware) and per-weight tiers, budgeted to a target bits-per-weight, and its evaluation in the trained format; `lean_stdm_parity.py` the parity hidden in the LOW tier.

`lean_nvfp4.py`, `lean_train_qat_nvfp4*.py`, `lean_pbs_nvfp4*.py` the NVFP4 mode (Appendix G).

`lean_archs.py` MobileNet-V2, EfficientNet-B0, ViT-B/16, Swin-T (Appendix H).

`attnbreaker_core.py`, `lean_llm_attnbreaker_w.py`, `lean_llm_showcase.py`, `lean_llm_showcase_codesel.py` AttentionBreaker (GenBFA) re-implemented on the INT8 weights with a weight-level code; `_codesel` adds the choice of code placement (`--code-score gradw|attacker`). `lean_llm_bf16.py` runs any of these scripts with the model in bfloat16.

`lean_llm_silentstriker_w.py` SilentStriker on the same models and code.

`lean_llm_harness.py`, `lean_llm_tasks.py`, `lean_llm_ppl_owt.py` six-task accuracy, MMLU, OpenWebText perplexity; `lean_llm_flipcheck.py` applies a saved flip list to any model and reports each flip's coverage under both placements.

`lean_overhead.py` per-epoch runtime and memory of the objective against a plain fine-tune.

4 Paper table and figure map

paper	scripts in `code/`
Table 1, Figure 3, Appendix B	`run_sensreg_all_models.sh`, `run_all_sensreg.sh`
Table 2	`run_lean_ablation.sh`, `run_lean_ablation2.sh`, `lean_truedamage.py`, `lean_train_gradnorm.py`
Appendix D (per-channel ablation)	`run_c4_cifar_pc.sh`, `run_c4_imnet_train.sh`, `run_c4_imnet_pbs.sh`, `run_c4_imnet_pcstep.sh`
Table 4	`run_lean_e3_pbsfour.sh`, `plot_compare_hessian.py`
Table 5	`run_lean_e3_seeds.sh`
Table 6, Appendix E	`run_lean_tbfa*.sh`, `run_fill_part2_tbfa.sh`
Table 7, Appendix F	`run_lean_codec.sh`, `run_fill_part1_codec.sh`, `run_c5_codeaware.sh`
Table 8, Figure 4, Appendix G	`run_lean_group5.sh`, `run_lean_mp9.sh`, `run_lean_nvfp4*.sh`, `run_trunc_sweep.sh`
Table 9, Appendix L	`hw/` (Section 7)
Table 10, Appendix J	`run_sensreg_llm.sh`, `run_lean_llm_full*.sh`, `run_lean_attnbreaker_w*.sh`, `run_lean_silentstriker*.sh`, `run_lean_qwen7b_*.sh`, `run_qwen7b_*.sh`, `run_llama8b_*.sh`, `run_lean_owt.sh`
Appendix C	`run_lean_inset*.sh`
Appendix H	`run_lean_archs*.sh`
Appendix I	`energy/run_all.sh`, `energy/lean_energy.py`
Appendix K	`run_lean_e6_logit.sh`, `run_lean_e7_band.sh`, `run_lean_e8_flipaware.sh`, `run_lean_archs_dw*.sh`, `run_lean_archs_clamp_all.sh`

Every `run_*.sh` starts with a comment saying what it reproduces, what arguments it takes (usually just the GPU index) and where it writes its log. Several of the longer ones were run in pieces over days, which is why some tables are covered by more than one script.

5 One model end to end

```
cd code
# LEAN fine-tune from the clean checkpoint, ResNet-18 CIFAR-10, lambda = 40
python lean_sensreg.py train --dataset cifar10 --arch resnet18 \
   --init artifacts/models/cifar10/resnet18/model_float32.pth --reg both --lam 40 --
    epochs 10 \
   --out artifacts/models/cifar10/resnet18/model_float32_sensreg_lam40.pth
# sensitivity distribution against the baseline's HIGH/MID thresholds
python lean_sensreg.py hist --dataset cifar10 --arch resnet18 \
   --ckpt artifacts/models/cifar10/resnet18/model_float32.pth \
   --ckpt2 artifacts/models/cifar10/resnet18/model_float32_sensreg_lam40.pth \
   --out sensreg_results/cifar10_resnet18/hist_lam40
# progressive bit search, selective 1% code, 100 to 500 flips
python pbs_four.py --dataset CIFAR10 --arch resnet18 --bits 8 --protect selcode \
   --covs 0.01 --flips 100,200,300,500 --label lam40
```

6 Language models

```
cd code
bash run_sensreg_llm.sh 0                    # Qwen2-0.5B: control and LEAN fine-tunes,
     HIGH/MID, INT4
bash run_lean_llm_full.sh 0                  # AttentionBreaker and six tasks, 0.5%
    code
bash run_lean_silentstriker_rank.sh 0        # SilentStriker, code by sign-flip damage,
     loss/logit split
bash run_lean_qwen7b_train.sh                # Qwen2.5-7B LEAN fine-tune (finite-
    difference tail term)
bash run_qwen7b_codesel_attack.sh 0          # Qwen2.5-7B per-channel, code by the
    attacker's ranking
bash run_llama8b_attsel_attack.sh 0 lam10_sum   # Llama-3.1-8B, one checkpoint per GPU
bash run_llama8b_s1.sh 1 lam10_sum gradw     # second attacker seed
bash run_lean_owt.sh 2                       # OpenWebText perplexity of the clean
    models
```

The attacks load the fine-tuned checkpoints from `ckpt_llm/<model>/<run>/`; the 7B and 8B models go through `lean_llm_bf16.py`.

7 Hardware

`hw/README.md` documents the loader in full. The flows, each a single command:

```
cd hw
python lean_pack.py --ckpt <model.pth> --bins <model.bins.pt> --tiers 8,6,4,3 --block
    32 --out vec_lean
python lean_hw_agent.py --ckpt <model.pth> --tiers 8,6,4,3 --block 32   # VCS
    simulation + DC synthesis
bash run_power_saif.sh <tag>          # simulation-driven power (SAIF)
bash fpga/run_fpga.sh                 # Vivado: INT8 read-out, loader, loader + checker
bash fpga/run_xsim.sh                 # post-route gate-level simulation, PASS/FAIL
bash fpga/run_fpga_accel.sh           # loader next to the 8x8 systolic accelerator
bash fpga/run_cycles.sh               # throughput and latency, cycle-accurate
bash fpga_soc/run_soc.sh              # Zynq UltraScale+ block design, AXI-Lite IP
```

The RTL (`rtl/lean_weight_loader.v`, `rtl/lean_unpack.v`) is bit-exact against the Python packer on every weight of ResNet-50; the testbench `tb/tb_lean_loader.v` also injects random bit flips into the packed image and reports which flips land in protected blocks and are flagged. Design Compiler targets a 45 nm library at 100 MHz; Vivado targets the ZCU102 (xczu9eg) and a Versal VM1802.

8 Energy model

`energy/` holds the Timeloop architecture (`arch_int8.yaml`), the row-stationary constraints, the mapper settings, the Accelergy component library and the per-layer workloads; `run_all.sh` runs every ResNet-50 layer at INT8 and in the tiered format and `lean_energy.py` sums the energy per inference and per activation reported in Appendix I.

9 Notes

Random seeds are fixed in every script through `--seed`, and the paper says which entries are single runs. Checkpoints and datasets are not included because of their size; every checkpoint used in the paper is produced by the scripts above from the public baselines, and the fine-tunes are short (ten epochs for vision, one epoch for language models), so regenerating them is cheaper than hosting them. If something does not run as described, the header comment of the script is the first place to look; the second is the log it writes.